

Lab Assignment 6

Object-Oriented Programming

Branch

B.E., 2nd Year – ENC



Submitted By:

Ryan Kapoor

Roll No. 1024150188

Batch: 2023

Q1

Implement a class Library that contains an array of Book objects. The Book class should have attributes like *title*, *author*, and *ISBN*. The Library class should provide the following methods:

(a) bool addNewBook(title, author, ISBN)

- Add a new book to the list
- Return type should be Boolean (true/false)
- Use pass-by-reference for arguments
- The function arguments and class attributes should have the same names
- Use the scope resolution operator to access class data members

(b) bool removeBooks(ISBN)

- Remove a book using ISBN
- Return Boolean (true/false)
- Use pass-by-reference
- Define this function outside the class using scope resolution operator

(c) displayDetails()

- Display all books in the

library Write a main function to:

- Add at least 5 books
- Remove 1 book
- Display all

books

Code:

```
#include <iostream>
using namespace std;
```

```
class Book {
public:
    string title;
    string author;
    string ISBN;
};
```

```
class Library {
public:
    Book books[100];
    int count = 0;
```

```
bool addNewBook(string &title, string &author, string &ISBN) {
```

```

        if(count >= 100)
            return false;

        books[count].title = title;
        books[count].author = author;
        books[count].ISBN = ISBN;

        count++;
        return true;
    }

    bool removeBooks(string &ISBN);

    void displayDetails() {
        for(int i = 0; i < count; i++) {
            cout << "Title: " << books[i].title << endl;
            cout << "Author: " << books[i].author << endl;
            cout << "ISBN: " << books[i].ISBN << endl;
            cout << endl;
        }
    }
};

bool Library::removeBooks(string &ISBN) {
    for(int i = 0; i < count; i++) {
        if(books[i].ISBN == ISBN) {
            for(int j = i; j < count - 1; j++) {
                books[j] = books[j + 1];
            }
            count--;
            return true;
        }
    }
    return false;
}

int main() {

    Library lib;

    string t1="C++", a1="Bjarne", i1="101";
    string t2="DSA", a2="Mark", i2="102";
    string t3="OS", a3="Galvin", i3="103";
    string t4="DBMS", a4="Korth", i4="104";
    string t5="CN", a5="Tanenbaum", i5="105";

    lib.addNewBook(t1, a1, i1);
    lib.addNewBook(t2, a2, i2);
    lib.addNewBook(t3, a3, i3);
    lib.addNewBook(t4, a4, i4);
    lib.addNewBook(t5, a5, i5);

    string rem = "103";
    lib.removeBooks(rem);

    lib.displayDetails();

    return 0;
}

```

```
PS C:\codes> cd "c:\codes\UN
● Title: C++
  Author: Bjarne
  ISBN: 101

  Title: DSA
  Author: Mark
  ISBN: 102

  Title: DBMS
  Author: Korth
  ISBN: 104

  Title: CN
  Author: Tanenbaum
  ISBN: 105

○ PS C:\codes\UNC401\OOPS\A2>
```

Q2

Implement Question 1 using constructors with the following modifications:

(a) Add:

- Default constructor
- Parameterized constructor
- Copy constructor
to initialize book details (*title, author, ISBN*)

(b) Use the `this` pointer instead of scope resolution operator to access class data members

(c) Create function `bool removeBooks(ISBN)` same as Question 1

(d) Create function `displayDetails()` same as Question 1

(e) Create array of objects using:

- **Initializer list**
- **Dynamic**

initialization

Code:

```
#include <iostream>
using namespace std;

class Book {
public:
    string title;
    string author;
    string ISBN;

    Book() {
        this->title = "";
        this->author = "";
        this->ISBN = "";
    }

    Book(string title, string author, string ISBN) {
        this->title = title;
        this->author = author;
        this->ISBN = ISBN;
    }

    Book(const Book &b) {
        this->title = b.title;
        this->author = b.author;
        this->ISBN = b.ISBN;
    }
};

class Library {
public:
    Book *books;
    int count;

    Library(int size) {
        books = new Book[size];
        count = 0;
    }

    bool addNewBook(Book &b) {
        books[count] = b;
        count++;
        return true;
    }

    bool removeBooks(string &ISBN);

    void displayDetails() {
        for(int i = 0; i < count; i++) {
            cout << "Title: " << books[i].title << endl;
```

```

        cout << "Author: " << books[i].author << endl;
        cout << "ISBN: " << books[i].ISBN << endl;
        cout << endl;
    }
}
};

```

```

bool Library::removeBooks(string &ISBN) {
    for(int i = 0; i < count; i++) {
        if(books[i].ISBN == ISBN) {
            for(int j = i; j < count - 1; j++) {
                books[j] = books[j + 1];
            }
            count--;
            return true;
        }
    }
    return false;
}

```

```

int main() {

    Book b1("C++", "Bjarne", "101");
    Book b2("DSA", "Mark", "102");
    Book b3("OS", "Galvin", "103");
    Book b4("DBMS", "Korth", "104");
    Book b5("CN", "Tanenbaum", "105");

    Book arr[3] = {
        Book("Math", "Author1", "201"),
        Book("Physics", "Author2", "202"),
        Book("Chemistry", "Author3", "203")
    };

    Library lib(10);

    lib.addNewBook(b1);
    lib.addNewBook(b2);
    lib.addNewBook(b3);
    lib.addNewBook(b4);
    lib.addNewBook(b5);

    for(int i = 0; i < 3; i++) {
        lib.addNewBook(arr[i]);
    }

    string rem = "103";
    lib.removeBooks(rem);

    lib.displayDetails();

    delete[] lib.books;

    return 0;
}

```

```
PS C:\codes\UNC401\OOPS\A2> cd ..
Title: C++
Author: Bjarne
ISBN: 101

Title: DSA
Author: Mark
ISBN: 102

Title: DBMS
Author: Korth
ISBN: 104

Title: CN
Author: Tanenbaum
ISBN: 105

Title: Math
Author: Author1
ISBN: 201

Title: Physics
Author: Author2
ISBN: 202

Title: Chemistry
Author: Author3
ISBN: 203
```

Q3

Create a class Account with the following attributes:

- account number (*const long type*)
- transaction ID (*long type*)
- transaction type (*credit/debit — string*)

- **balance** (*double type*)

The class should provide the following methods:

(a) long depositAmount(to, from, amount)

- “to” and “from” are account numbers
- **Return the transaction ID**

(b) long creditAmount(to, from, amount)

- **Similar to depositAmount**
- **Return the transaction ID**

(c) void displayDetails()

- **Display account number, balance, and transaction history**
- **Make this a const function**

(d) In all functions, make all arguments const

Write a main function to:

- **Create at least 5 Account objects**
- **Demonstrate credit and debit operations**

Code:

```
#include <iostream>
using namespace std;
```

```
class Account {
public:
```

```
    const long accountNumber;
    long transactionID;
    string transactionType;
    double balance;
```

```
    static long counter;
```

```
    Account(long acc, double bal) : accountNumber(acc) {
        balance = bal;
        transactionID = 0;
        transactionType = "None";
    }
}
```

```
long depositAmount(const long to, const long from, const double amount) {
    if(accountNumber == to) {
        balance += amount;
        transactionType = "Credit";
        transactionID = ++counter;
        return transactionID;
    }
    return -1;
}
```



```

    }

    long creditAmount(const long to, const long from, const double amount) {
        if(accountNumber == from && balance >= amount) {
            balance -= amount;
            transactionType = "Debit";
            transactionID = ++counter;
            return transactionID;
        }
        return -1;
    }

    void displayDetails() const {
        cout << "Account No: " << accountNumber << endl;
        cout << "Balance: " << balance << endl;
        cout << "Last Transaction ID: " << transactionID << endl;
        cout << "Transaction Type: " << transactionType << endl;
        cout << endl;
    }
};

long Account::counter = 1000;

int main() {

    Account a1(101, 5000);
    Account a2(102, 6000);
    Account a3(103, 7000);
    Account a4(104, 8000);
    Account a5(105, 9000);

    a1.depositAmount(101, 102, 1000);
    a2.creditAmount(101, 102, 500);

    a3.depositAmount(103, 104, 2000);
    a4.creditAmount(104, 103, 1000);

    a5.depositAmount(105, 101, 1500);

    a1.displayDetails();
    a2.displayDetails();
    a3.displayDetails();
    a4.displayDetails();
    a5.displayDetails();

    return 0;
}

```

```
...
Account No: 101
Balance: 6000
Last Transaction ID: 1001
Transaction Type: Credit

Account No: 102
Balance: 5500
Last Transaction ID: 1002
Transaction Type: Debit

Account No: 103
Balance: 9000
Last Transaction ID: 1003
Transaction Type: Credit

Account No: 104
Balance: 8000
Last Transaction ID: 0
Transaction Type: None

Account No: 105
Balance: 10500
Last Transaction ID: 1004
Transaction Type: Credit
```

⊗ 0 ⚠ 0  Amazon Q Indexing completed. ☕

Q4

Write a program to add data objects of two different classes using friend function.

Code:

```
#include <iostream>
using namespace std;

class B;

class A {
    int x;
public:
```

```

    A(int a) {
        x = a;
    }

    friend int add(A, B);
};

class B {
    int y;
public:
    B(int b) {
        y = b;
    }

    friend int add(A, B);
};

int add(A obj1, B obj2) {
    return obj1.x + obj2.y;
}

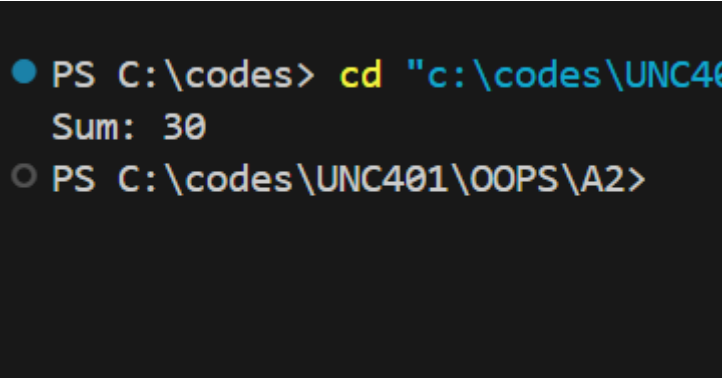
int main() {

    A obj1(10);
    B obj2(20);

    int result = add(obj1, obj2);
    cout << "Sum: " << result << endl;

    return 0;
}

```



```

● PS C:\codes> cd "c:\codes\UNC401\OOPS\A2"
Sum: 30
○ PS C:\codes\UNC401\OOPS\A2>

```

Q5

Define a class **Complex** with properties (*real and imaginary*) and implement:

(a) Parameterized constructor and copy constructor to initialize object values

(b) void display() to display the complex number

(c) void sum(Complex, Complex)

- Add two complex numbers
- Implement this as a friend

function

Code:

```
#include <iostream>
using namespace std;

class Complex {
    float real;
    float imag;

public:
    Complex(float r, float i) {
        real = r;
        imag = i;
    }

    Complex(const Complex &c) {
        real = c.real;
        imag = c.imag;
    }

    void display() {
        cout << real << " + " << imag << "i" << endl;
    }

    friend void sum(Complex, Complex);
};

void sum(Complex c1, Complex c2) {
    float r = c1.real + c2.real;
    float i = c1.imag + c2.imag;

    cout << "Sum: " << r << " + " << i << "i" << endl;
}

int main() {
```

```
Complex c1(2.5, 3.5);
Complex c2(1.5, 2.5);

Complex c3 = c1;

cout << "First Complex: ";
c1.display();

cout << "Second Complex: ";
c2.display();

cout << "Copied Complex: ";
c3.display();

sum(c1, c2);

return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE

PS C:\codes\UNC401\OOPS\A2>

- `cd`
First Complex: $2.5 + 3.5i$
Second Complex: $1.5 + 2.5i$
Copied Complex: $2.5 + 3.5i$
Sum: $4 + 6i$
- PS C:\codes\UNC401\OOPS\A2>

